

4 – REPETITION

This provides the structure to process code blocks multiple times. Hence, you have the opportunity to write programs that might not have been logistically possible. E.g. Adding the numbers from one to one million.

There are no do-while loop nor are there the normal for loop. Python supports the break and continue keywords.

4.a – While statements

The while loop can be considered a conditional iteration. The statements in the loop body are processed continually whilst the assertion is true and processing ceases when the assertion becomes false.

```
while «assertion»:  
... Body of while
```

```
>>> x=1  
>>> sum=0  
>>> while x < 100:  
...     sum += x  
...     x += 1  
...  
>>> print(sum)  
4950
```

4.b.1 – For loop

This is a determinate loop i.e., the number of iterations is determined by an iterable such as a string, list, tuple or range() object.

4.b.1.1 – Using an enumerable

Enumerable includes strings, lists, sets, tuples and dictionaries.

```
>>>a = [3, 5, 7, 8]
>>>for x in a:
...   print(x, end=' ')
...
3 5 7 8 >>>
```

```
>>> name = 'Narendra'
>>> for a in name:
...   print(a, end=' ')
...
N a r e n d r a >>>
```

4.b.1.2 – Using the range object

The range() function returns a range object which is an immutable sequence of numbers. A for loop is normally used to iterate this sequence. The range function takes one mandatory and two optional arguments.

```
range([«start=0»], «end», [«step=1»])
```

```
>>>for x in range(3, 10, 2):
...   print(x, end=' ')
...
3, 5, 7, 9 >>>
```

```
# if you are not using the value, then the linter
# will complain about un-used variables. The under
# score (referred to as a 'discard') prevents this
# often annoying reporting.
>>>for _ in range( 10 ):
...   print('Hello', end=' ')
Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello
```

4.b.1.3 – Using the Enumerate Function

The `enumerate()` function takes a collection (a sequence or mapping) and adds a counter and then return an enumerate object.

```
enumerate(«collection», [«start=0»])
```

```
>>> profs='ilia yin arben narendra mehrdad'.split()
>>> #process all the items
... for pos, name in enumerate(profs):
...   print(f'{pos} {name}')
...
0 ilia
1 yin
2 arben
3 narendra
4 mehrdad
```

```
# you can start the count at a particular integer
>>> profs='ilia yin arben narendra mehrdad'.split()
```

```
>>> for pos, name in enumerate(profs, start=2):
...     print(f'{pos} {name}')
...
2 ilia
3 yin
4 arben
5 narendra
6 mehrdad
```

```
# you can iterate a dictionary
>>> profs = {'ilia': 'COMP304', 'yin': 'COMP306', 'arben': 'COMP305',
'narendra': 'COMP216', 'mehrdad': 'COMP225'}
>>> for pos, name in enumerate(profs):
...     print(f'{pos}: {name}')
...
0: ilia
1: yin
2: arben
3: narendra
4: mehrdad
```

```
# a better way to iterate a dictionary is to use the .items() method of the
dictionary class
>>>
>>> for prof, course in profs.items():
...     print(f'{prof.rjust(12)}: {course}')
...
      ilia: COMP304
      yin: COMP306
      arben: COMP305
narendra: COMP216
mehrdad: COMP225
```

4.b.1.4 – Nested Loop

A loop within another loop is referred to as a nested loop.

```
distros = 'ubuntu mint centos'.split()
arch = '32-bit 64-bit'.split()
for d in distros:
    for a in arch:
        print(d, a)
    print('_____')
```

```
ubuntu 32-bit
ubuntu 64-bit
```

```
_____
mint 32-bit
mint 64-bit
```

```
_____
centos 32-bit
centos 64-bit
_____
```

While Loop

This is a loop that will keep repeating itself until the while condition becomes false. If there is no logic to change the condition of the while, then you will have a non-terminating loop (infinite loop)

```
n = 1                                #loop initializer
while n < 100:                       #loop checker
    n += 1                           #loop updater
```

For Loop

For loops give you more control than while loops. You can loop through anything that is iterable. e.g. a string, a range, a list, a dictionary or tuple.

```
fruits = ['apple', 'banana', 'orange']
for f in fruits:          #fruits is a list
    print(f)
```

_ in a For Loop

If the value your for loop is iterating through, e.g. the number in the range, or the item in the list is not needed, you can replace it with an underscore.

```
for _ in range(100):
    #Do something 100 times.
```

break

This keyword allows you to break free of the loop. You can use it in a for or while loop to terminate when a particular condition is detected.

```
scores = [34, 67, 99, 105]
for s in scores:
    if s > 100:
        print('Invalid')
        break
    print(s)
```

continue

This keyword allows you to skip this iteration of the loop and go to the next. The loop will still continue, but it will start from the top.

```
n = 1
while n < 100:
    if n % 2 == 0:
        continue
```

```
print(n)
#Prints all the odd numbers
```

pass

This keyword allows you to place a nothing statement (one that does not do anything) where a statement is needed. It is often used when you are not sure or you have not decided on the implementation details. It can be used for the body of a class, a method, the body of a loop or a block in an if statement.

```
for x in range(20):
    if x % 3 == 0:
        pass          #a nothing statement
    else:
        print(x)
```

Infinite Loops

Sometimes, the condition you are checking to see if the loop should continue never becomes false. In this case, the loop will continue for eternity (or until your computer stops it). This is more common with while loops.

```
while 5 > 1:
    print("I'm a survivor")
```